CSE 5350 - Computer Architecture II

Fall 2026 | Programming Assignment

CAT2: Computer Architecture 2 "VM"

Due: See Canvas | Demo: Required (details will be announced)

Overview

CAT2 is a small virtual machine used to explore processor organization and performance. The Python 3 starter package provides the basic CAT2 assembler/VM structure, memory and register model, a shared architecture definition, and starter benchmark files. Your task is to extend that framework through Parts 1-4 below.

Starter-code expectations: Use the supplied Python 3 starter files. Architecture parameters are centralized in cat2.py; do not create separate, conflicting hard-coded word/register/opcode sizes in the assembler and executor. If you want to use another programming language, discuss it with the instructor/TA first.

Architecture assumptions for this assignment

- Use the classic 5-stage pipeline model: IF, ID, EX, MEM, WB.
- For Part 2 hazard analysis, assume no forwarding. Implement stalls based on when source operands become available.
- The VM must be able to show (dump) register and memory contents and relevant metadata needed for each part.
- Add or modify CAT2 benchmark programs as needed to demonstrate correctness and performance.

Part 1 - Store and basic instrumentation

- Implement the CAT Store (st) instruction.
- Instrument the VM to report the number of instructions executed, total clock cycles, and total memory references. Distinguish instruction and data references where appropriate.
- Use a benchmark to demonstrate that Store writes the correct value to memory and that subsequent loads observe the stored value.

Part 2 - Pipeline hazards and branch prediction

- Implement data-hazard detection using a scoreboard (or equivalent mechanism) for the 5-stage pipeline model.
- Detect RAW dependencies and determine the exact number of stall cycles required under the no-forwarding assumption.
- Implement control-hazard handling for branch/control-flow instructions and apply the appropriate stalls/flushes.
- Implement a 1-bit branch predictor for CAT conditional branches.
- During execution, show where each relevant hazard occurs (source assembly and/or instruction address), why it occurs, the scoreboard state, and how many cycles are stalled.
- Show the branch-predictor table/state as branch instructions execute, including prediction outcomes and mispredictions.

Part 3 - L1 cache memory

Implement an L1 cache in addition to RAM and support each required configuration below.

Organization	Instruction cache	Data cache
Direct mapped, unified	2 words x 4 lines	shared
Direct mapped, unified	4 words x 4 lines	shared
Direct mapped, unified	2 words x 8 lines	shared
Split I/D	2 words x 2 lines	2 words x 2 lines
Split I/D	4 words x 2 lines	4 words x 4 lines
2-way set associative, unified	2 words x 8 total lines	shared

Optional/Bonus: add a simple L2 cache configured as 8 words x 8 lines.

Timing assumptions: L1 = 1 cycle, L2 = 4 cycles, main memory = 25 cycles.

Write-policy assumption for grading consistency: use write-through + write-allocate. You may assume an ideal write buffer for the simplified timing model unless otherwise directed by the instructor.

- For every configuration, report instruction references, data references, hits, misses, hit rate, cache contents, and total performance/clock-cycle impact.
- Test with the supplied benchmark(s) and at least one benchmark that is large enough to exercise the cache meaningfully.

Part 4 - Vector / superscalar CAT instructions

- Add a vector-add instruction that performs element-by-element addition of two vectors.
- Add a vector-sum instruction that sums the elements of a vector in one CAT instruction.
- Modify the 5-stage execution/timing model as needed to represent these new instructions and explain your timing assumptions.
- Write a small CAT benchmark that adds two 1 x N vectors. The count N must be less than 32.
- Demonstrate the resulting vector and the vector-sum behavior during your demo.

Submission and demo

- Submit the requested source files and benchmark programs through Canvas. Do not submit generated executables/output files unless Canvas specifically asks for them.
- Your program should run with Python 3 and should not require editing hard-coded architecture values in multiple files.
- Be prepared to demo the assignment to the TA/instructor and explain your implementation choices, measurements, hazards, cache behavior, and vector operations.